

Sales Performance Analysis Report

1. Introduction

This project focuses on analyzing sales performance using SQL, Python (Pandas), and Power BI.

The main objective is to understand:

- Product performance
- Customer purchase behavior
- Revenue distribution
- Sales trends across categories

2. Tools & Technologies Used

Tool	Purpose
SQL Server	Data extraction, transformation, aggregation
Python (Pandas)	Data cleaning and preprocessing
Power BI	Data visualization and dashboard creation

3. Data Preparation & Cleaning

3.1 Data Sources

Data was collected from: <https://www.datawithbaraa.com/wiki/sql>

3.2 Data Cleaning (Python – Pandas)

- Removed records with null order dates
- Ensured correct data types
- Removed inconsistencies and missing values
- Prepared clean datasets for SQL analysis

Sales_details

	order_number	product_key	customer_key	order_date	shipping_date	due_date	sales_amount	quantity	price
0	SO54496	282	5400	3/16/2013	3/23/2013	3/28/2013	25	1	25
1	SO54496	289	5400	3/16/2013	3/23/2013	3/28/2013	5	1	5
2	SO54496	259	5400	3/16/2013	3/23/2013	3/28/2013	2	1	2
3	SO54497	174	9281	3/16/2013	3/23/2013	3/28/2013	22	1	22
4	SO54497	280	9281	3/16/2013	3/23/2013	3/28/2013	9	1	9
...	...	...	...	...	...	...	...	...	...
60393	SO54494	259	11279	3/16/2013	3/23/2013	3/28/2013	2	1	2
60394	SO54495	289	3750	3/16/2013	3/23/2013	3/28/2013	5	1	5
60395	SO54495	244	3750	3/16/2013	3/23/2013	3/28/2013	35	1	35
60396	SO54495	248	3750	3/16/2013	3/23/2013	3/28/2013	50	1	50
60397	SO54495	166	3750	3/16/2013	3/23/2013	3/28/2013	9	1	9

60379 rows × 9 columns

```
Sales_details.to_csv("Cleaned_Sales_Details.csv", index=False)
```

Product_Details

	product_key	product_id	product_number	product_name	category_id	category	subcategory	maintenance	cost	product_line	start_date
0	1	210	FR-R92B-58	HL Road Frame - Black- 58	CO_RF	Components	Road Frames	Yes	0	Road	7/1/2003
1	2	211	FR-R92R-58	HL Road Frame - Red- 58	CO_RF	Components	Road Frames	Yes	0	Road	7/1/2003
2	3	348	BK-M82B-38	Mountain-100 Black- 38	BI_MB	Bikes	Mountain Bikes	Yes	1898	Mountain	7/1/2011
3	4	349	BK-M82B-42	Mountain-100 Black- 42	BI_MB	Bikes	Mountain Bikes	Yes	1898	Mountain	7/1/2011
4	5	350	BK-M82B-44	Mountain-100 Black- 44	BI_MB	Bikes	Mountain Bikes	Yes	1898	Mountain	7/1/2011
...	...	...	...	...	...	...	...	...	...	...	...
290	291	530	TT-T092	Touring Tire Tube	AC_TT	Accessories	Tires and Tubes	Yes	2	Touring	7/1/2013
291	292	473	VE-C304-L	Classic Vest- L	CL_VE	Clothing	Vests	No	24	Other Sales	7/1/2013
292	293	472	VE-C304-M	Classic Vest- M	CL_VE	Clothing	Vests	No	24	Other Sales	7/1/2013
293	294	471	VE-C304-S	Classic Vest- S	CL_VE	Clothing	Vests	No	24	Other Sales	7/1/2013
294	295	477	WB-H098	Water Bottle - 30 oz.	AC_BC	Accessories	Bottles and Cages	No	2	Other Sales	7/1/2013

295 rows × 11 columns

```
Product_Details.to_csv("cleaned_product_details.csv", index=False)
```

4. SQL Analysis Approach

- Combine product attributes with sales data
- Ensure data accuracy

4.1 Data Extraction and Integration

- The first step involved combining product and sales data using a LEFT JOIN between the product and sales tables based on the Product Key

```
select
  p.product_key,
  p.product_name,
  p.category,
  p.subcategory,
  p.cost,
  s.order_number,
  s.order_date,
  s.customer_key,
  s.sales_amount,
  s.quantity
from cleaned_product_details p
left join Cleaned_Sales_Details s
on s.product_key = p.product_key
```

4.2 Aggregation Product Level Metrics

- Total Orders → Number of unique orders per product
- Total Sales → Total revenue generated by each product
- Total Quantity Sold → Total number of units sold
- Total Customers → Unique customers per product
- Product Lifespan → Time between first and last order (in months)

```

/* 2. Aggregates product-level metrics:
   - total orders
   - total sales
   - total quantity sold
   - total customers (unique)
   - lifespan (in months) */
, Product_Aggregation as (
select
product_key,
product_name,
category,
subcategory,
cost,
Count(distinct order_number) as Total_Orders,
sum(sales_amount) as Total_Sales,
sum(quantity) as Total_Quantity_Sold,
count(distinct customer_key) as Total_Customer,
datediff(month, min(order_date), max(order_date) ) as LifeSpan,
max(order_date) as Last_OrderDate
from base_Query
group by
product_key,
product_name,
category,
subcategory,
cost
)

```

4.3 Sales Segment

High ($\geq 500K$)	57.42%
Medium ($\geq 100K$)	33.57%
Low ($< 100K$)	9.02%

```

--segment products by revenue to identify high-performance, mid-range, or low-performance
select
product_key,
product_name,
category,
subcategory,
cost,
Total_Orders,
Total_Sales,
Total_Quantity_Sold,
Total_Customer,
LifeSpan,
Last_OrderDate,
case when Total_Sales >= 500000 then 'HIGH Performance'
      when Total_Sales >= 100000 then 'MID-Range Performance'
      else 'LOW Performance'
end Product_Segment,

```

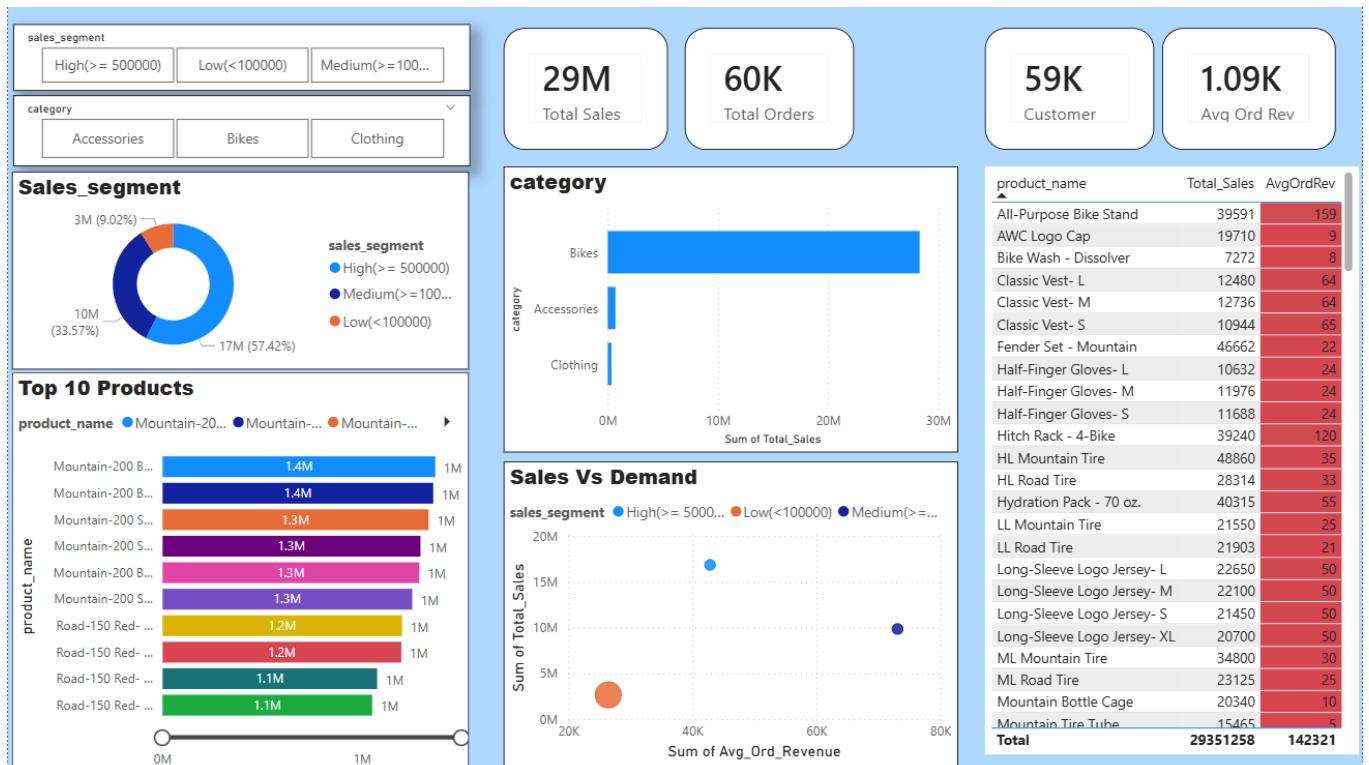
4.4 Valuable KPI's

- Recency in Month
- Average Order Revenue
- Avg Monthly Revenue

```
--4. calculates valuable KPIs:
-- recency (months since last sale)
datediff(month, Last_OrderDate, getdate()) as Recency_In_Month,
-- Average Order Value
case when Total_Orders = 0 then 0
     else Total_Sales / Total_Orders
end Avg_Ord_Revenue,
--Average Monthly Revenue
case when LifeSpan = 0 then 0
     else Total_Sales / LifeSpan
end Avg_Monthly_Revenue
from Product_Aggregation
```

Recency_In_Month	Avg_Ord_Revenue	Avg_Monthly_Revenue
169	3375	15034
169	3375	13806
169	3375	18409
169	3375	16031
169	3400	16433
169	3400	12981
169	3400	13883
169	3400	11127
169	3578	83784
169	3578	100482
169	3578	90046
169	3578	87959
169	3578	100184
157	755	2068
157	759	2169
157	751	2909
157	763	2521
157	752	2598
157	754	2043
157	757	2370
157	758	2900
157	755	1920
157	761	2561

5. DashBoard in Power BI



6. Business Recommendations

1. Focus on High-Performing Products
 - Ensure stock availability.
 - Promote premium bike models.
2. Improve Low-Performing Categories
 - Apply discounts on clothing.
 - Bundle products (Bike + Accessories).
3. Increase Average Order Value
 - Introduce combo offers.
 - Offer loyalty discounts.
4. Optimize Inventory Planning
 - Reduce stock for slow-moving items.
 - Increase inventory for high-demand SKUs.